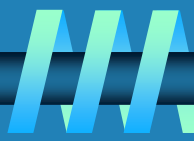
Iterative Improvement



Algorithm design technique for solving optimization problems

- **Start with a feasible solution**
- **Repeat the following step until no improvement can be found:**
  - **change the current feasible solution to a feasible solution with a better value of the objective function**
- **Return the last feasible solution as optimal**

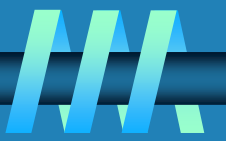
Note: Typically, a change in a current solution is "small" (*local search*)

Major difficulty: Local optimum vs. global optimum



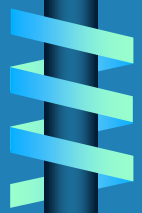
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Important Examples



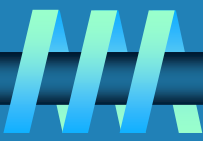
- simplex method
- Ford-Fulkerson algorithm for maximum flow problem
- maximum matching of graph vertices
- Gale-Shapley algorithm for the stable marriage problem

- local search heuristics



A. Levitin “Introduction to the Design & Analysis of Algorithms,” 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Linear Programming



Linear programming **(LP) problem is to optimize a linear function of several variables subject to linear constraints:**

maximize (or minimize) $c_1 x_1 + ... + c_n x_n$

subject to $a_{i\,1} x_1 + ... + a_{in} x_n \leq (\text{or} \geq \text{or} =)\ b_i,\ i = 1,...,m$

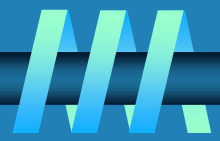
$x_1 \geq 0, \ ... \ , x_n \geq 0$

The function $z = c_1 x_1 + ... + c_n x_n$ **is called the *objective function*; constraints** $x_1 \geq 0, \ ... \ , x_n \geq 0$ **are called *nonnegativity constraints***



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example

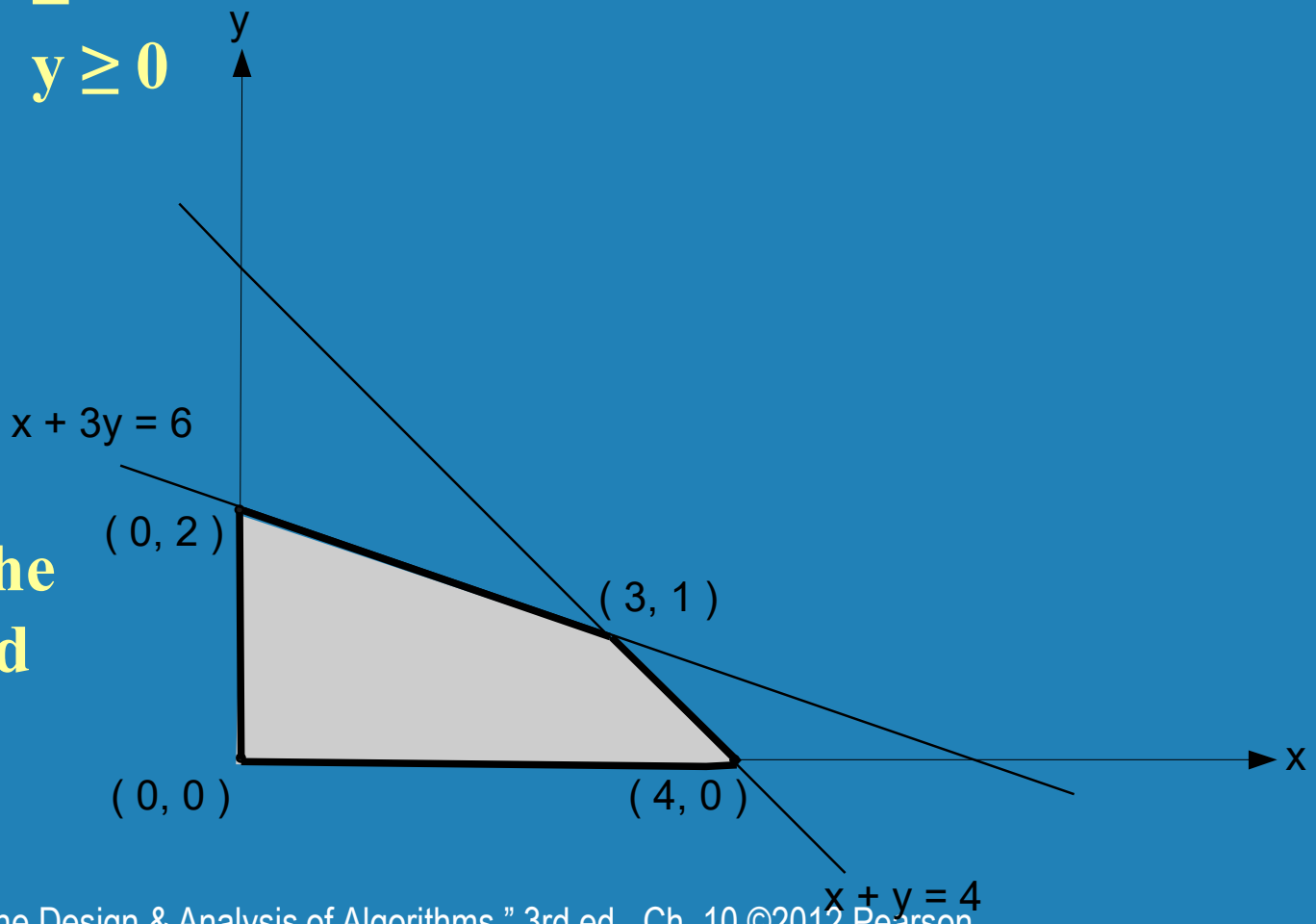


maximize $3x + 5y$

subject to $x + y \leq 4$

$x + 3y \leq 6$

$x \geq 0,\ y \geq 0$



Feasible region **is the set of points defined by the constraints**



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

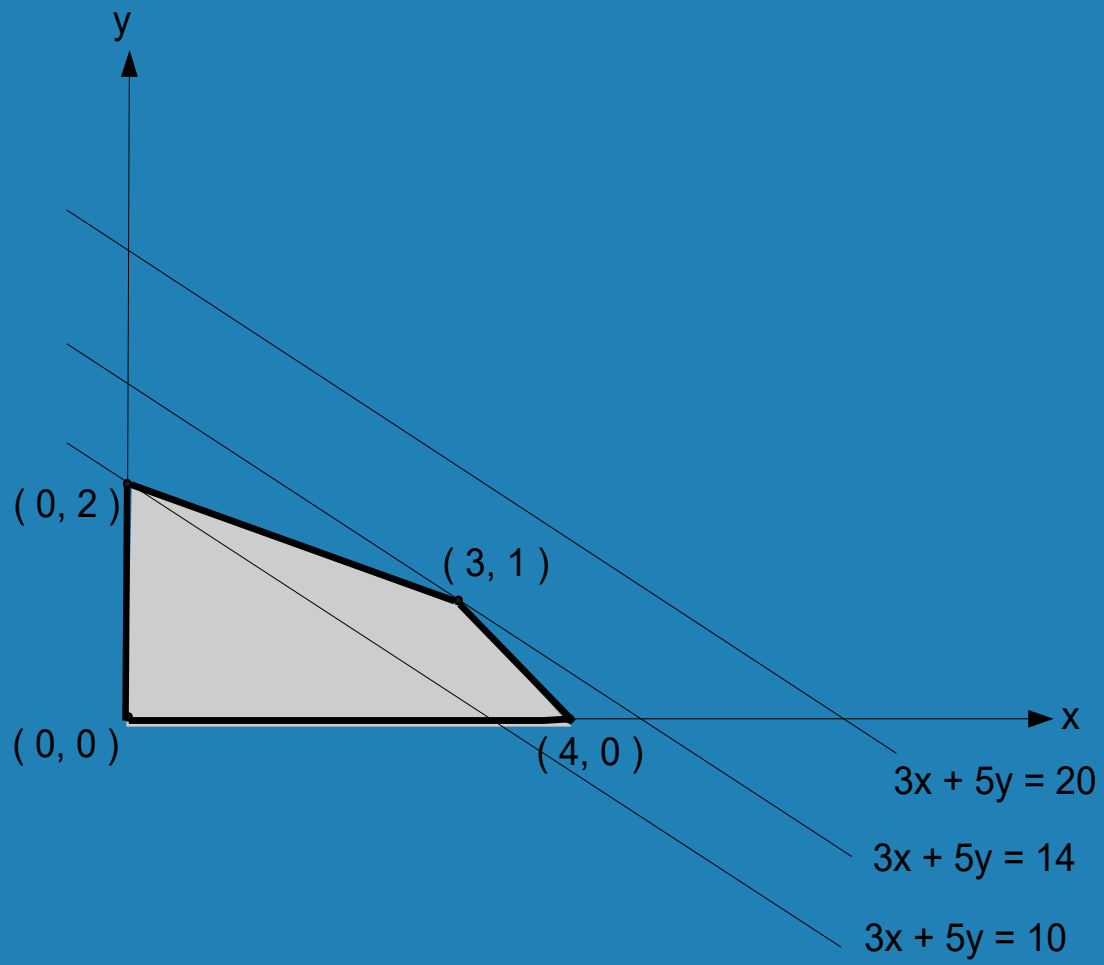
Geometric solution

maximize $3x + 5y$

subject to $x + y \leq 4$

$x + 3y \leq 6$

$x \geq 0,\ y \geq 0$

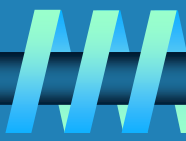


Optimal solution: x = 3, y = 1

<u>Extreme Point Theorem</u> **Any LP problem with a nonempty bounded feasible region has an optimal solution; moreover, an optimal solution can always be found at an *extreme point* of the problem's feasible region.**

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

3 possible outcomes in solving an LP problem

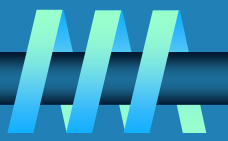


- has a finite optimal solution, which may no be unique
- *unbounded*: the objective function of maximization (minimization) LP problem is unbounded from above (below) on its feasible region
- *infeasible*: there are no points satisfying all the constraints, i.e. the constraints are contradictory



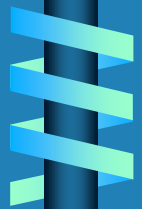
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

The Simplex Method



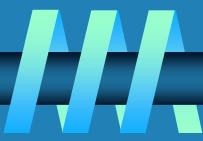
- The classic method for solving LP problems; one of the most important algorithms ever invented
- Invented by George Dantzig in 1947
- Based on the iterative improvement idea:

  Generates a sequence of adjacent points of the problem's feasible region with improving values of the objective function until no further improvement is possible



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Standard form of LP problem



- must be a maximization problem
- all constraints (except the nonnegativity constraints) must be in the form of linear equations
- all the variables must be required to be nonnegative

Thus, the general linear programming problem in standard form with m constraints and n unknowns ($n \geq m$) is

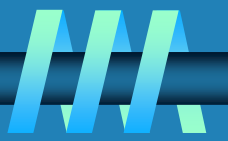
$$\text{maximize} \quad c_1 x_1 + \ldots + c_n x_n$$
$$\text{subject to} \quad a_{i\,1} x_1 + \ldots + a_{in} x_n = b_i, \quad i = 1,\ldots,m,$$
$$x_1 \geq 0, \ldots, x_n \geq 0$$

Every LP problem can be represented in such form

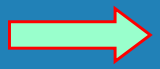


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example

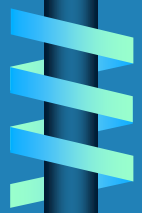


maximize $3x + 5y$
subject to $x + y \le 4$
$x + 3y \le 6$
$x \ge 0, \quad y \ge 0$



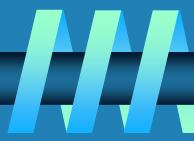
maximize $3x + 5y + 0u + 0v$
subject to $x + y + u = 4$
$x + 3y + v = 6$
$x \ge 0, \ y \ge 0, \ u \ge 0, \ v \ge 0$

Variables u and v, transforming inequality constraints into equality constrains, are called *slack variables*



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Basic feasible solutions



A *basic solution* to a system of m linear equations in n unknowns ($n \geq m$) is obtained by setting $n - m$ variables to 0 and solving the resulting system to get the values of the other m variables. The variables set to 0 are called *nonbasic*; the variables obtained by solving the system are called *basic*.

A basic solution is called *feasible* if all its (basic) variables are nonnegative.

Example $x + y + u = 4$

$x + 3y + v = 6$

(0, 0, 4, 6) is basic feasible solution

(x, y are nonbasic; u, v are basic)



There is a 1-1 correspondence between extreme points of LP's feasible region and its basic feasible solutions.

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

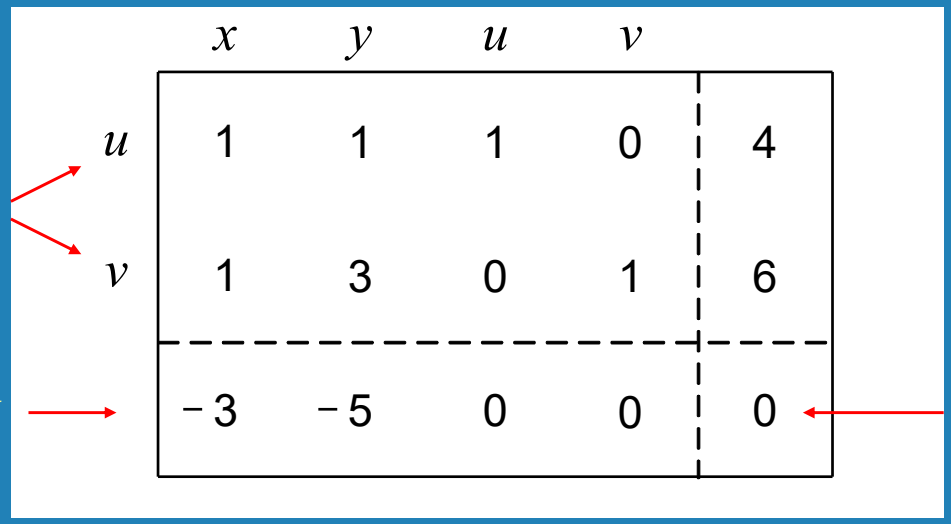
Simplex Tableau

maximize $z = 3x + 5y + 0u + 0v$

subject to $x + y + u = 4$

$x + 3y + v = 6$

$x \geq 0,\ y \geq 0,\ u \geq 0,\ v \geq 0$



basic variables

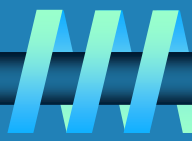
basic feasible solution (0, 0, 4, 6)

objective row

value of z at (0, 0, 4, 6)

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Outline of the Simplex Method



Step 0 [Initialization] Present a given LP problem in standard form and set up initial tableau.

Step 1 [Optimality test] If all entries in the objective row are nonnegative — stop: the tableau represents an optimal solution.

Step 2 [Find entering variable] Select (the most) negative entry in the objective row. Mark its column to indicate the entering variable and the pivot column.

Step 3 [Find departing variable] For each positive entry in the pivot column, calculate the θ-ratio by dividing that row's entry in the rightmost column by its entry in the pivot column. (If there are no positive entries in the pivot column — stop: the problem is unbounded.) Find the row with the smallest θ-ratio, mark this row to indicate the departing variable and the pivot row.

Step 4 [Form the next tableau] Divide all the entries in the pivot row by its entry in the pivot column. Subtract from each of the other rows, including the objective row, the new pivot row multiplied by the entry in the pivot column of the row in question. Replace the label of the pivot row by the variable's name of the pivot column and go back to Step 1.



Example of Simplex Method Application



maximize $z = 3x + 5y + 0u + 0v$

subject to $x + y + u = 4$

$x + 3y + v = 6$

$x \geq 0,\ y \geq 0,\ u \geq 0,\ v \geq 0$

	x	y	u	v	
u	1	1	1	0	4
← v	1	3	0	1	6
	−3	−5	0	0	0

↑

	x	y	u	v	
← u	$\frac{2}{3}$	0	1	$-\frac{1}{3}$	2
y	$\frac{1}{3}$	1	0	$\frac{1}{3}$	2
	$-\frac{4}{3}$	0	0	$\frac{5}{3}$	10

↑

	x	y	u	v	
x	1	0	$\frac{3}{2}$	$-\frac{1}{2}$	3
y	0	1	$-\frac{1}{2}$	$\frac{1}{2}$	1
	0	0	2	1	14

basic feasible sol.
(0, 0, 4, 6)

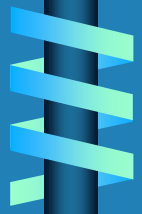
$z = 0$

basic feasible sol.
(0, 2, 2, 0)

$z = 10$

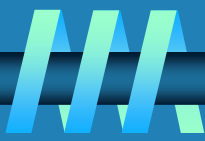
basic feasible sol.
(3, 1, 0, 0)

$z = 14$

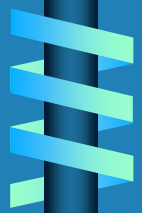


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Notes on the Simplex Method

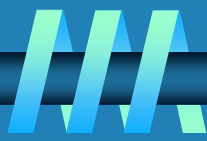


- Finding an initial basic feasible solution may pose a problem
- Theoretical possibility of cycling
- Typical number of iterations is between m and $3m$, where m is the number of equality constraints in the standard form
- Worse-case efficiency is exponential
- More recent *interior-point algorithms* such as *Karmarkar's algorithm* (1984) have polynomial worst-case efficiency and have performed competitively with the simplex method in empirical tests



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

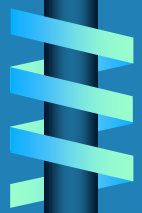
Maximum Flow Problem



Problem of maximizing the flow of a material through a transportation network (e.g., pipeline system, communications or transportation networks)

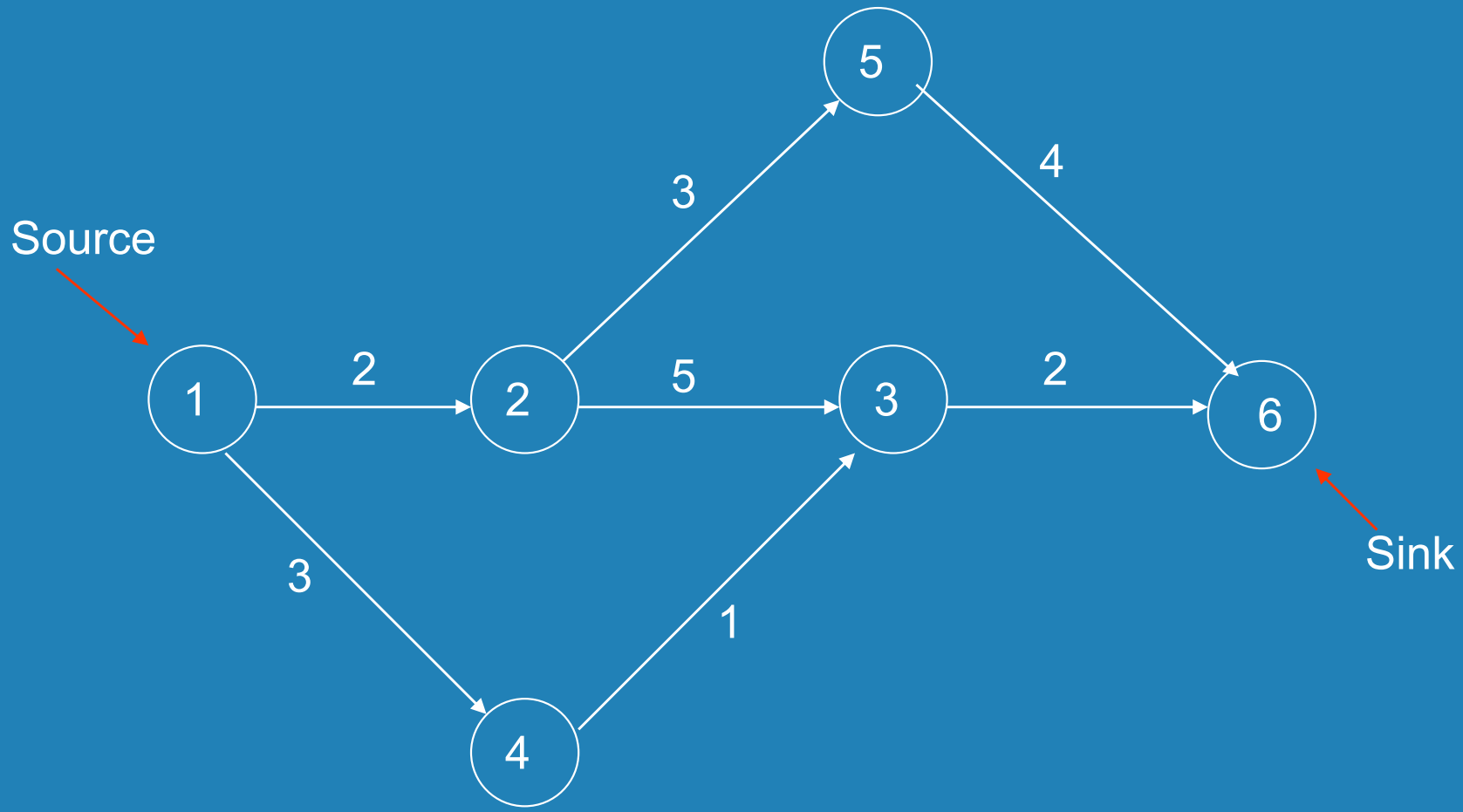
Formally represented by a connected weighted digraph with *n* vertices numbered from 1 to *n* with the following properties:

- **contains exactly one vertex with no entering edges, called the *source* (numbered 1)**
- **contains exactly one vertex with no leaving edges, called the *sink* (numbered *n*)**
- **has positive integer weight u_{ij} on each directed edge (*i*,*j*), called the *edge capacity*, indicating the upper bound on the amount of the material that can be sent from *i* to *j* through this edge**



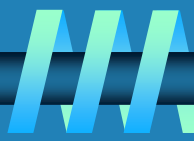
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example of Flow Network



A. Levitin “Introduction to the Design & Analysis of Algorithms,” 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Definition of a Flow



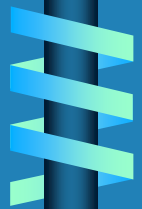
A *flow* is an assignment of real numbers x_{ij} to edges (i,j) of a given network that satisfy the following:

- *flow-conservation requirements*
  The total amount of material entering an intermediate vertex must be equal to the total amount of the material leaving the vertex

$$\sum_{j:\,(j,i)\,\epsilon\,E} x_{ji} = \sum_{j:\,(i,j)\,\epsilon\,E} x_{ij} \quad \text{for } i = 2,3,\ldots, n\text{-}1$$

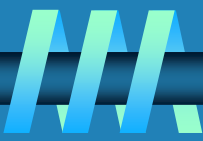
- *capacity constraints*

$$0 \le x_{ij} \le u_{ij} \quad \text{for every edge } (i,j) \in E$$



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Flow value and Maximum Flow Problem

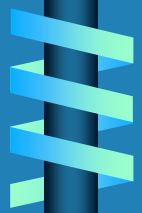


Since no material can be lost or added to by going through intermediate vertices of the network, the total amount of the material leaving the source must end up at the sink:

$$\sum_{j:\,(1,j)\,\in E} x_{1j} \quad = \quad \sum_{j:\,(j,n)\,\in E} x_{jn}$$

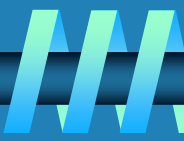
The *value* of the flow is defined as the total outflow from the source (= the total inflow into the sink).

The *maximum flow problem* is to find a flow of the largest value (maximum flow) for a given network.



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Maximum-Flow Problem as LP problem

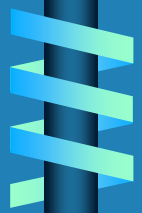


Maximize $v = \sum_{j:\,(1,j) \in E} x_{1j}$

subject to

$$\sum_{j:\,(j,i) \in E} x_{ji} - \sum_{j:\,(i,j) \in E} x_{ij} = 0 \quad \text{for } i = 2, 3, \ldots, n\text{-}1$$

$$0 \le x_{ij} \le u_{ij} \quad \text{for every edge } (i,j) \in E$$



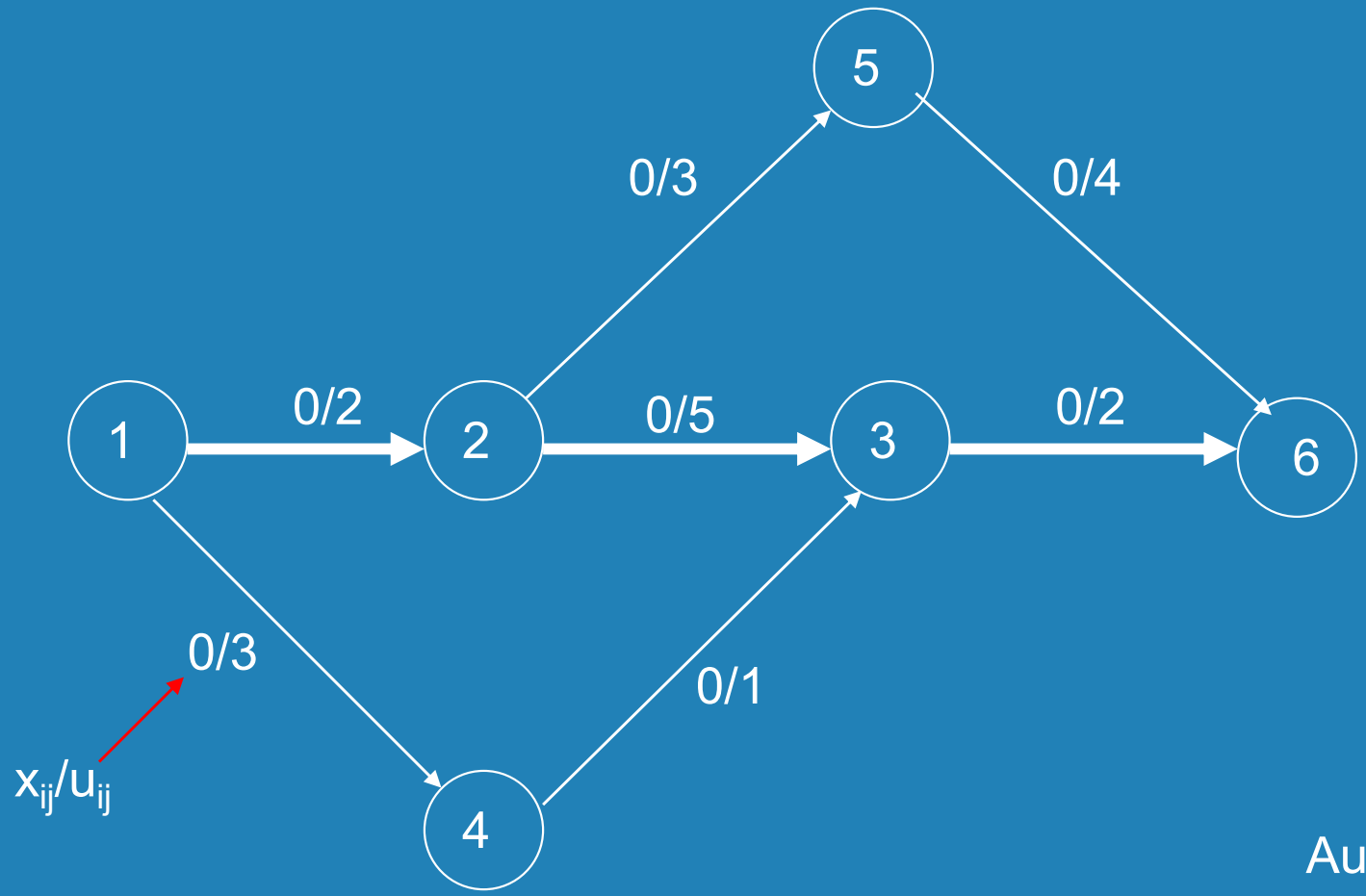
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Augmenting Path (Ford-Fulkerson) Method

- Start with the zero flow ($x_{ij} = 0$ for every edge)
- On each iteration, try to find a *flow-augmenting path* from source to sink, which a path along which some additional flow can be sent
- If a flow-augmenting path is found, adjust the flow along the edges of this path to get a flow of increased value and try again
- If no flow-augmenting path is found, the current flow is maximum

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

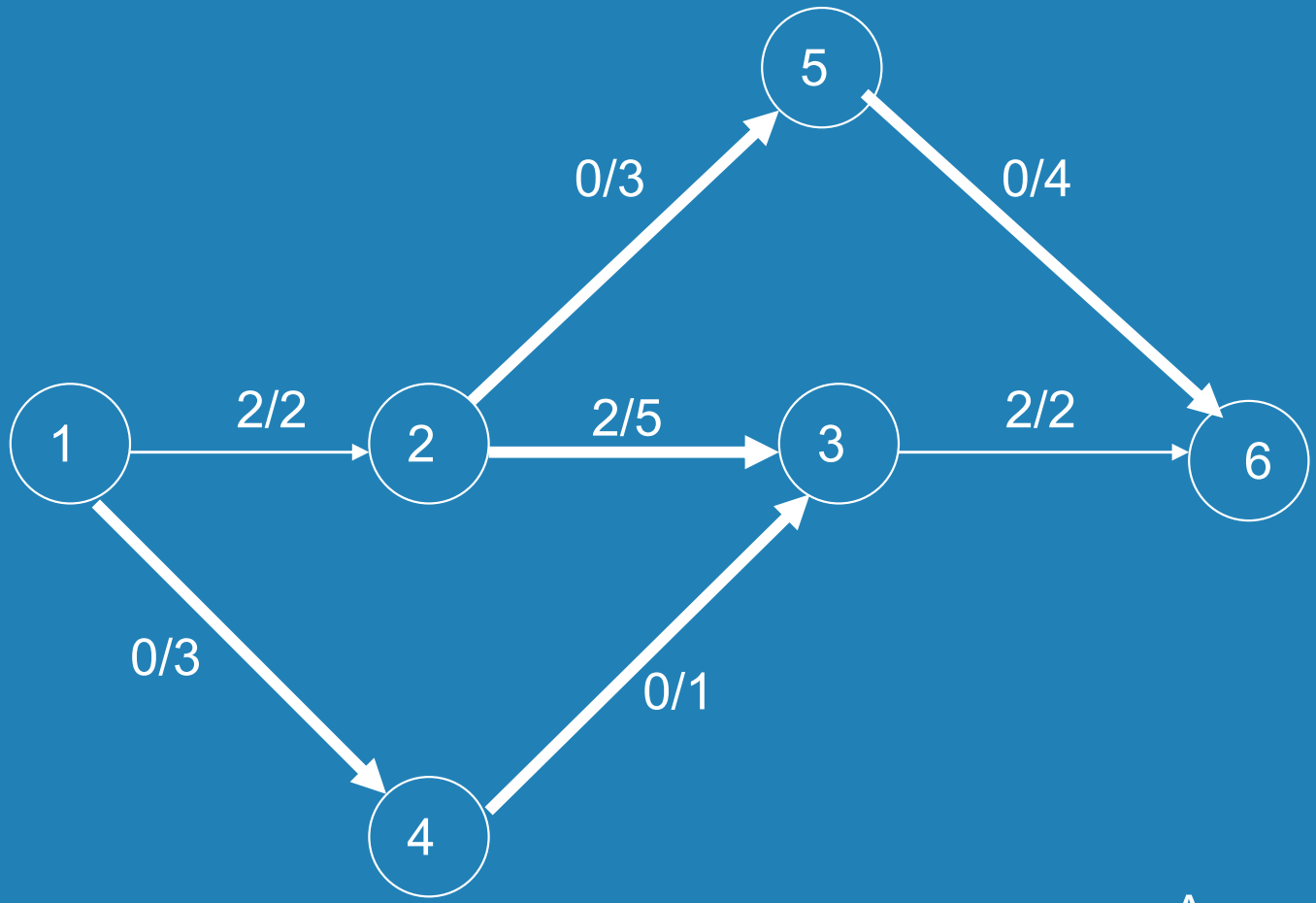
Example 1



Augmenting path:
1→2 →3 →6

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example 1 (cont.)

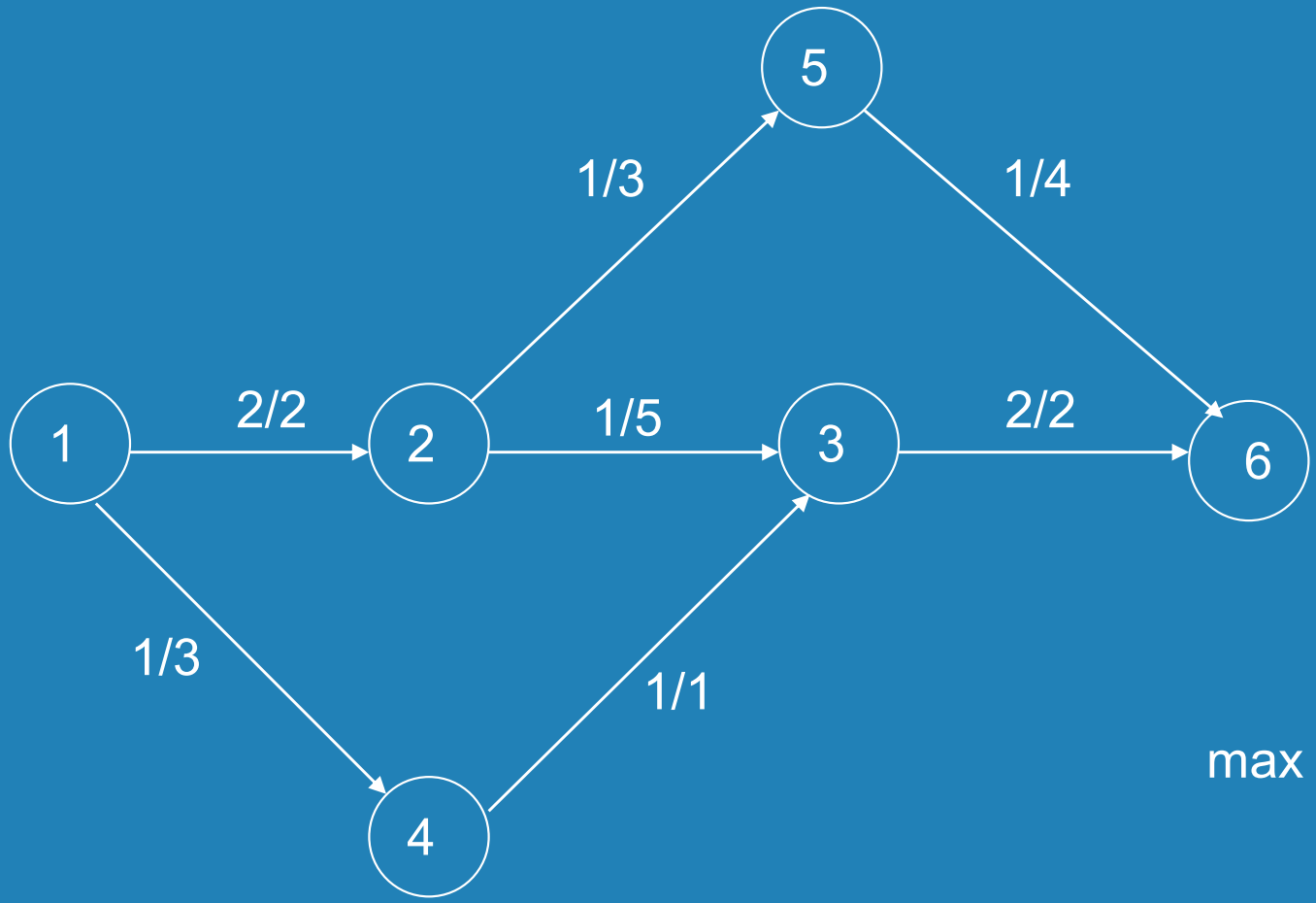


Augmenting path:
1 →4 →3←2 →5 →6

backward edge

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example 1 (maximum flow)



max flow value = 3

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Finding a flow-augmenting path



To find a flow-augmenting path for a flow x, consider paths from source to sink in the underlying undirected graph in which any two consecutive vertices i,j are either:

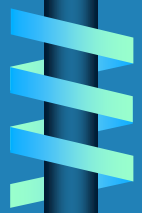
- connected by a directed edge (i to j) with some positive unused capacity $r_{ij} = u_{ij} - x_{ij}$
  - known as *forward edge* ($\rightarrow$)

OR

- connected by a directed edge (j to i) with positive flow x_{ji}
  - known as *backward edge* ($\leftarrow$)

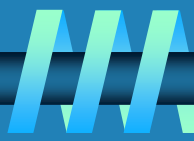
If a flow-augmenting path is found, the current flow can be increased by r units by increasing x_{ij} by r on each forward edge and decreasing x_{ji} by r on each backward edge, where

$$r = \min \{r_{ij} \text{ on all forward edges}, x_{ji} \text{ on all backward edges}\}$$

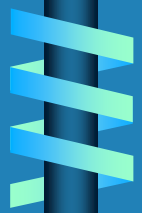


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Finding a flow-augmenting path (cont.)



- Assuming the edge capacities are integers, r is a positive integer
- On each iteration, the flow value increases by at least 1
- Maximum value is bounded by the sum of the capacities of the edges leaving the source; hence the augmenting-path method has to stop after a finite number of iterations
- The final flow is always maximum, its value doesn't depend on a sequence of augmenting paths used

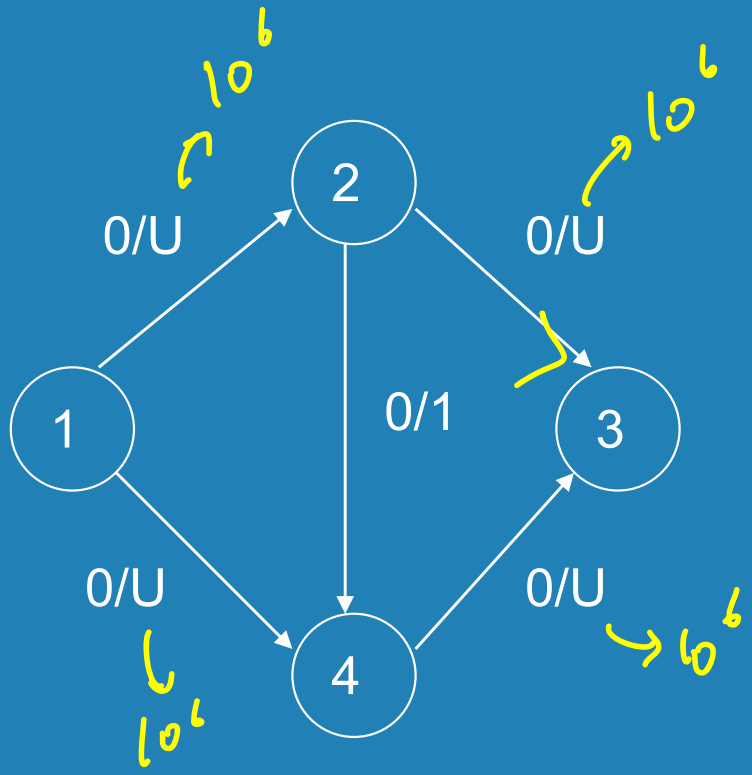


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Performance degeneration of the method

- The augmenting-path method doesn't prescribe a specific way for generating flow-augmenting paths
- Selecting a bad sequence of augmenting paths could impact the method's efficiency

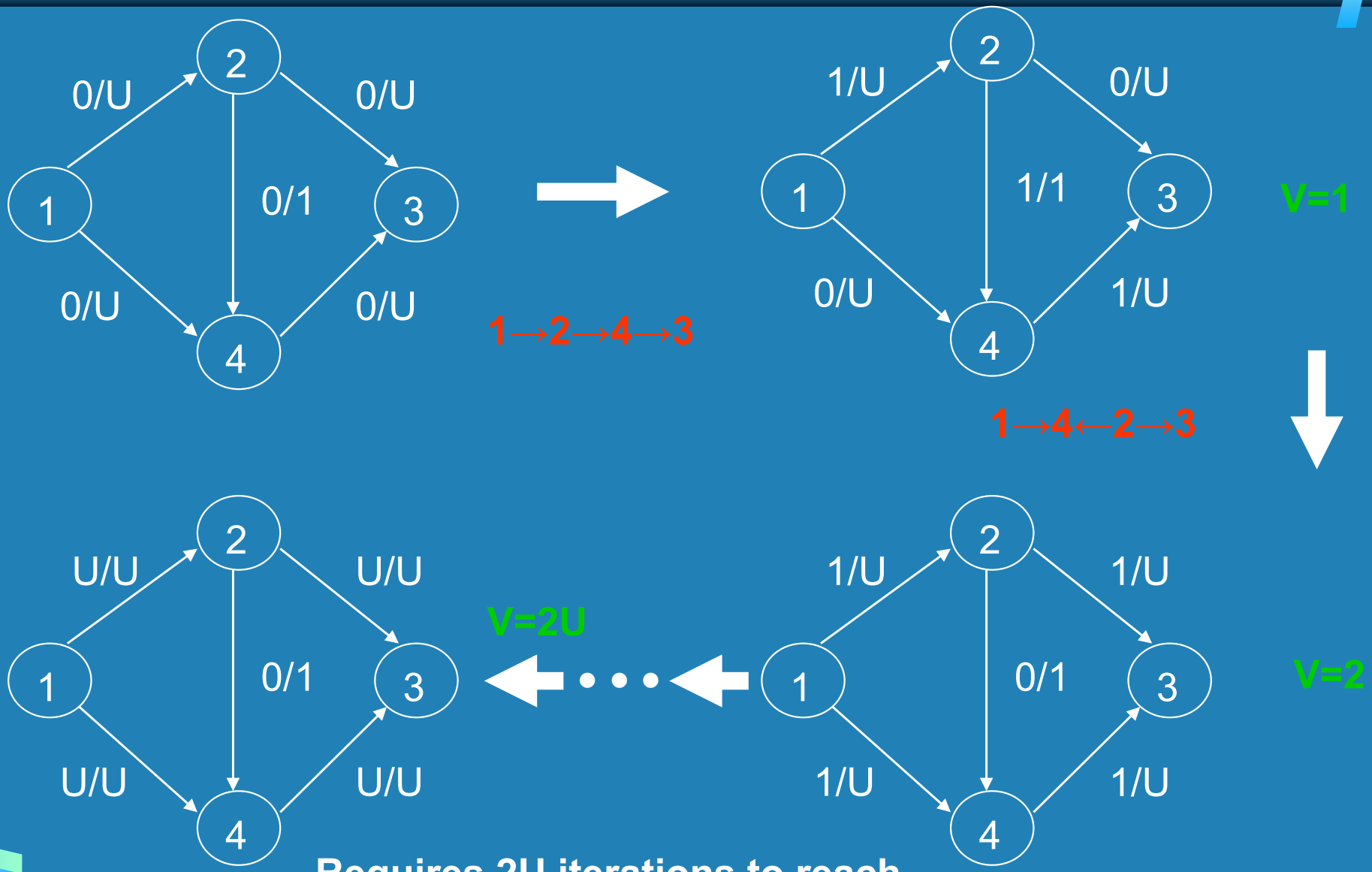
Example 2



U = large positive integer

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

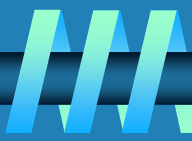
Example 2 (cont.)



Requires 2U iterations to reach maximum flow of value 2U

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Shortest-Augmenting-Path Algorithm



Generate augmenting path with the least number of edges by BFS as follows.

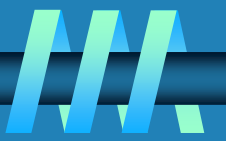
Starting at the source, perform BFS traversal by marking new (unlabeled) vertices with two labels:

- **first label – indicates the amount of additional flow that can be brought from the source to the vertex being labeled**
- **second label – indicates the vertex from which the vertex being labeled was reached, with "+" or "–" added to the second label to indicate whether the vertex was reached via a forward or backward edge**



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Vertex labeling

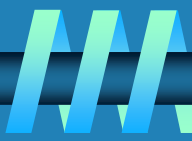


- The source is always labeled with ∞,-
- All other vertices are labeled as follows:
  - If unlabeled vertex j is connected to the front vertex i of the traversal queue by a directed edge from i to j with positive unused capacity $r_{ij} = u_{ij} - x_{ij}$ (forward edge), vertex j is labeled with l_j, i^+, where $l_j = \min\{l_i, r_{ij}\}$
  - If unlabeled vertex j is connected to the front vertex i of the traversal queue by a directed edge from j to i with positive flow x_{ji} (backward edge), vertex j is labeled l_j, i^-, where $l_j = \min\{l_i, x_{ji}\}$

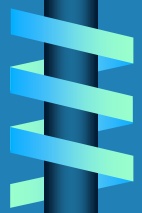


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Vertex labeling (cont.)

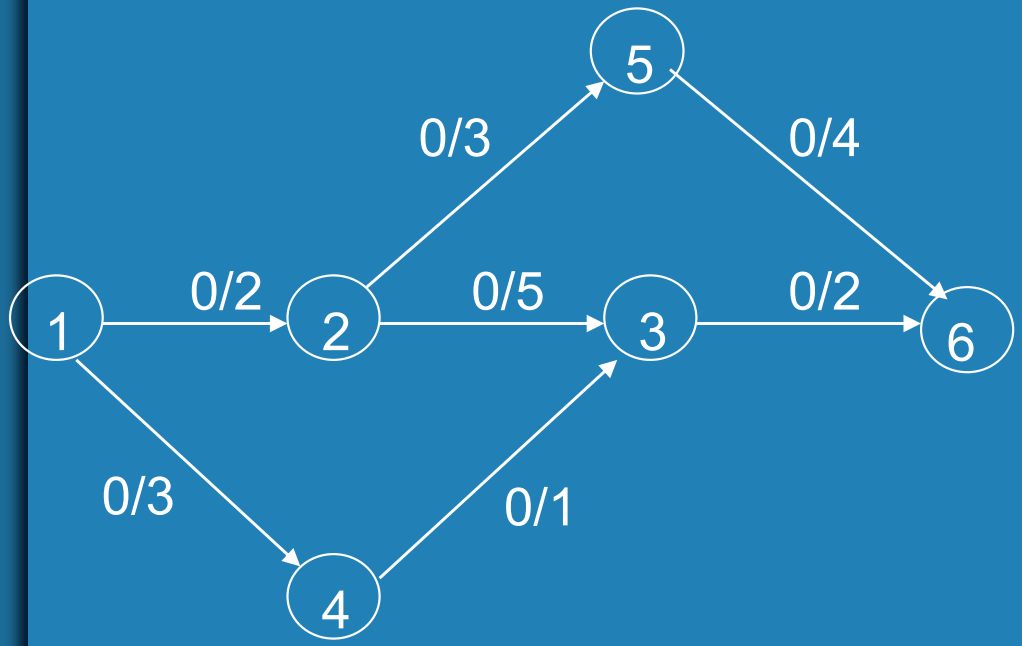


- If the sink ends up being labeled, the current flow can be augmented by the amount indicated by the sink's first label
- The augmentation of the current flow is performed along the augmenting path traced by following the vertex second labels from sink to source; the current flow quantities are increased on the forward edges and decreased on the backward edges of this path
- If the sink remains unlabeled after the traversal queue becomes empty, the algorithm returns the current flow as maximum and stops

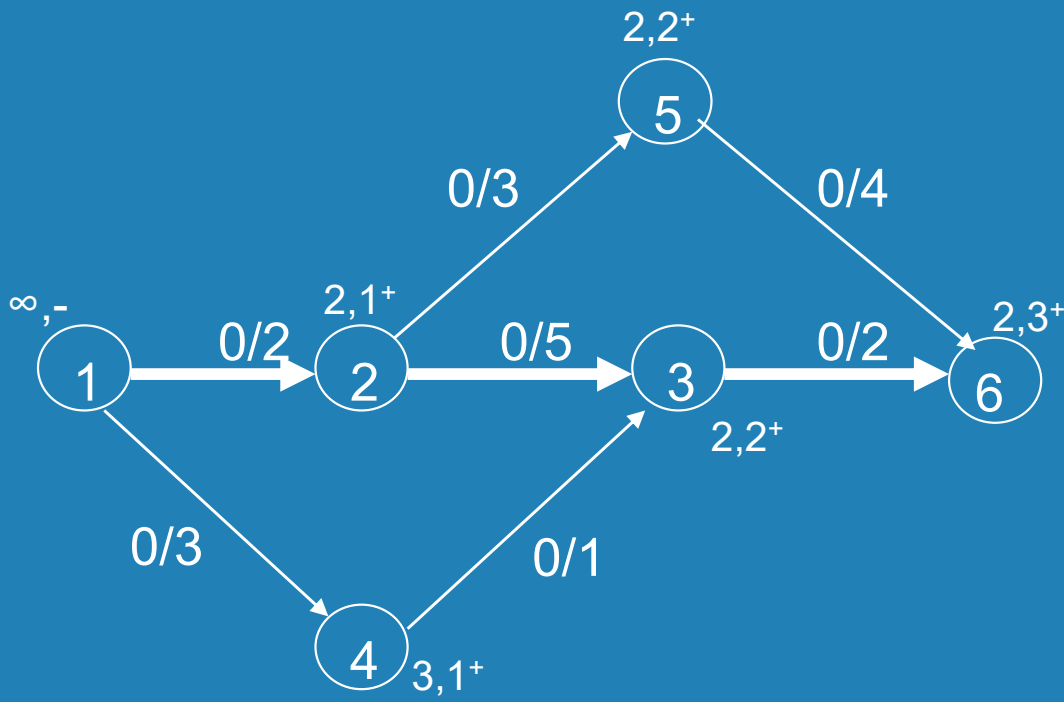


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: Shortest-Augmenting-Path Algorithm



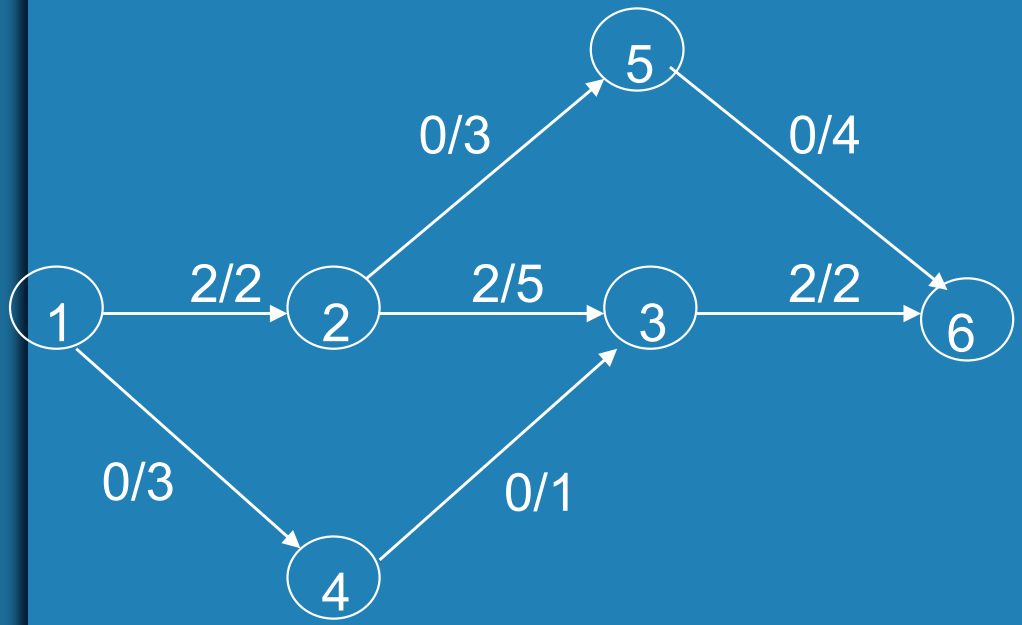
Queue: 1 2 4 3 5 6



Augment the flow by 2 (the sink's first label) along the path 1→2→3→6

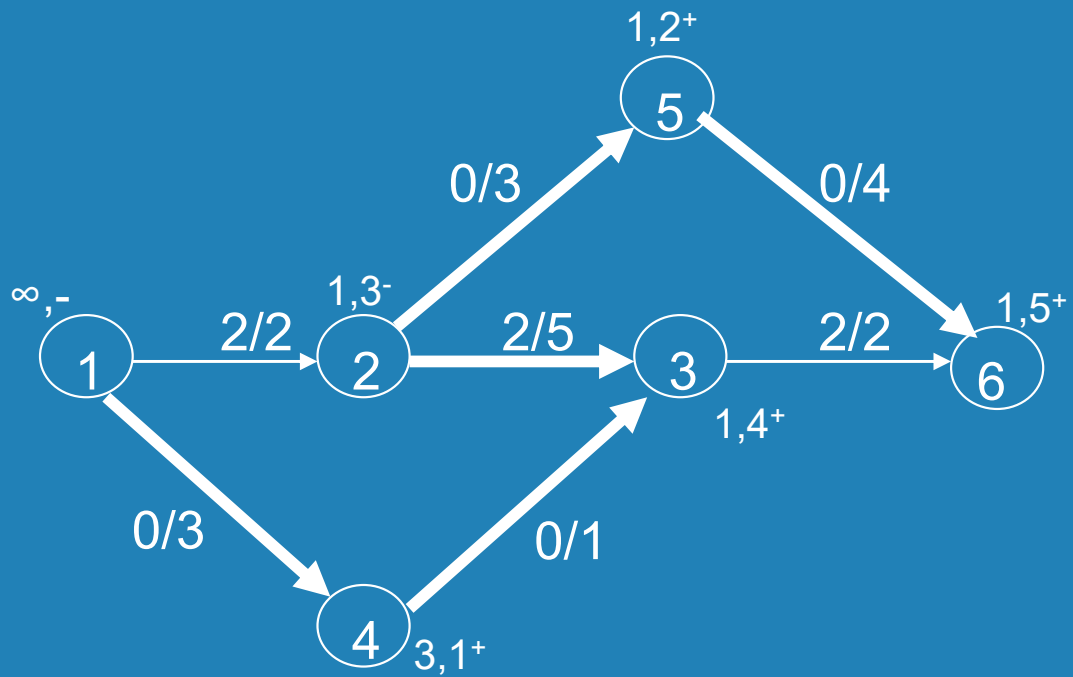
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)



Queue: 1 4 3 2 5 6

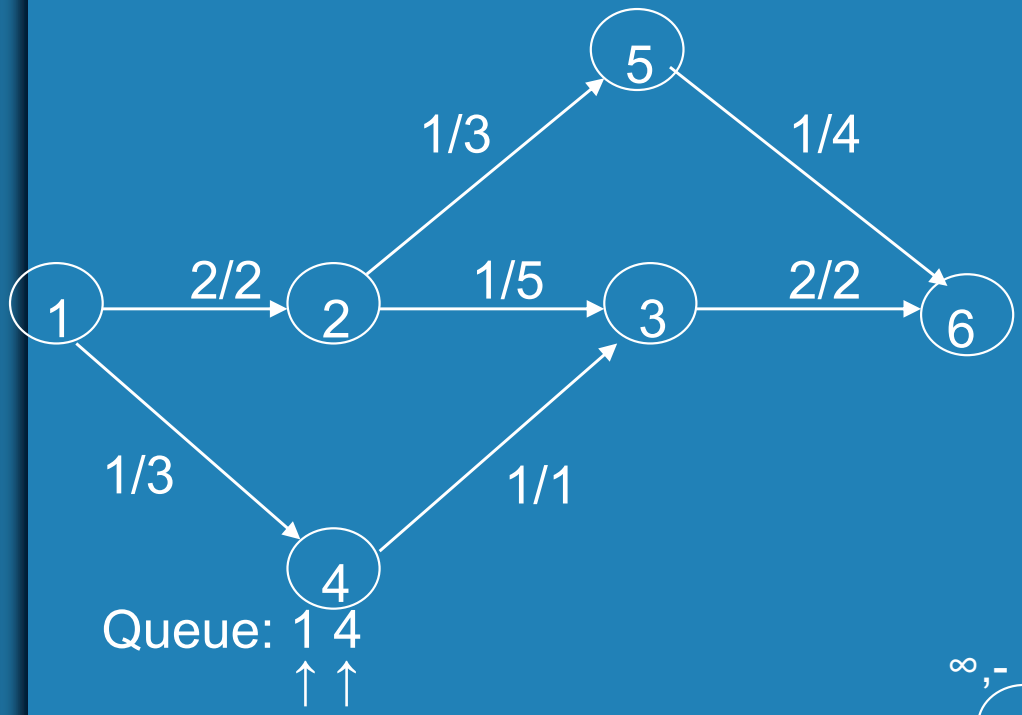
↑ ↑ ↑ ↑ ↑

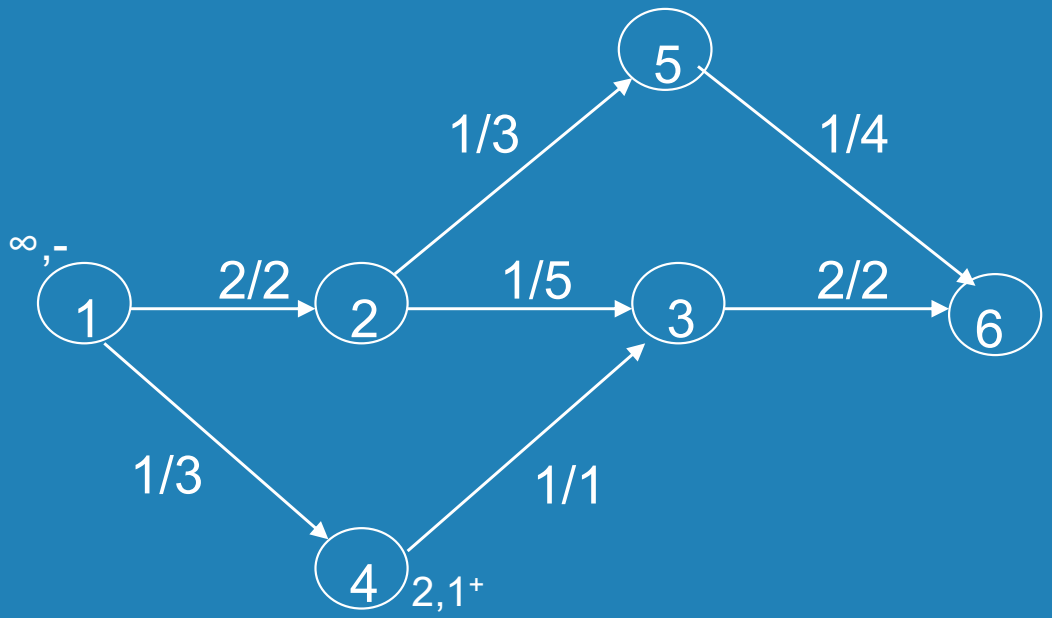


Augment the flow by 1 (the sink's first label) along the path 1→4→3←2→5→6

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)





No augmenting path (the sink is unlabeled) the current flow is maximum

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

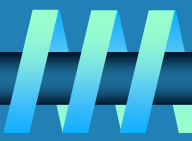
Shortest-augmenting-path algorithm

Input: A network with single source 1, single sink n, and
positive integer capacities u_{ij} on its edges (i, j)

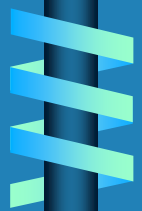
Output: A maximum flow x

assign $x_{ij} = 0$ to every edge (i, j) in the network
label the source with $\infty, -$ and add the source to the empty queue Q
while not $Empty(Q)$ **do**
    $i \leftarrow Front(Q)$; $Dequeue(Q)$
    for every edge from i to j **do** //forward edges
        if j is unlabeled
            $r_{ij} \leftarrow u_{ij} - x_{ij}$
            if $r_{ij} > 0$
                $l_j \leftarrow \min\{l_i, r_{ij}\}$; label j with l_j, i^+
                $Enqueue(Q, j)$
    for every edge from j to i **do** //backward edges
        if j is unlabeled
            if $x_{ji} > 0$
                $l_j \leftarrow \min\{l_i, x_{ji}\}$; label j with l_j, i^-
                $Enqueue(Q, j)$
    if the sink has been labeled
        //augment along the augmenting path found
        $j \leftarrow n$ //start at the sink and move backwards using second labels
        while $j \neq 1$ //the source hasn't been reached
            if the second label of vertex j is i^+
                $x_{ij} \leftarrow x_{ij} + l_n$
            else //the second label of vertex j is i^-
                $x_{ji} \leftarrow x_{ji} - l_n$
            $j \leftarrow i$
        erase all vertex labels except the ones of the source
        reinitialize Q with the source
return x //the current flow is maximum

Time Efficiency

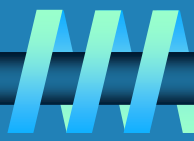


- The number of augmenting paths needed by the shortest-augmenting-path algorithm never exceeds $nm/2$, where n and m are the number of vertices and edges, respectively
- Since the time required to find shortest augmenting path by breadth-first search is in $O(n+m)=O(m)$ for networks represented by their adjacency lists, the time efficiency of the shortest-augmenting-path algorithm is in $O(nm^2)$ for this representation
- More efficient algorithms have been found that can run in close to $O(nm)$ time, but these algorithms don't fall into the iterative-improvement paradigm



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

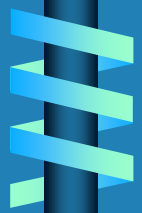
Definition of a Cut



Let X be a set of vertices in a network that includes its source but does not include its sink, and let $\overline{X}$, the complement of X, be the rest of the vertices including the sink. The *cut* induced by this partition of the vertices is the set of all the edges with a tail in X and a head in $\overline{X}$.

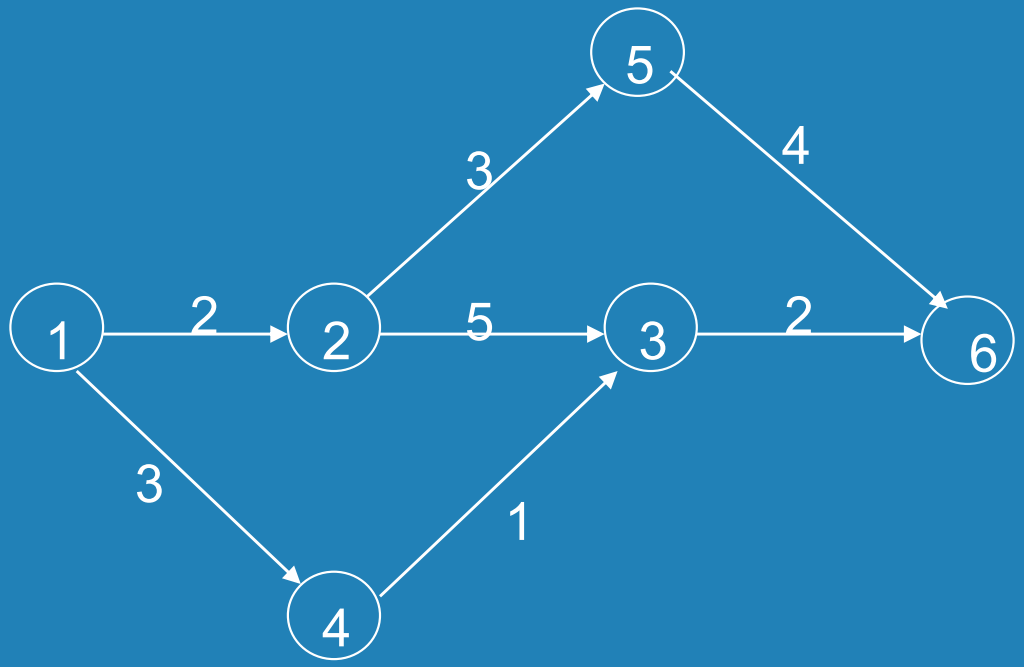
Capacity of a cut is defined as the sum of capacities of the edges that compose the cut.

- We'll denote a cut and its capacity by $C(X,\overline{X})$ and $c(X,\overline{X})$
- Note that if all the edges of a cut were deleted from the network, there would be no directed path from source to sink
- *Minimum cut* is a cut of the smallest capacity in a given network



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Examples of network cuts



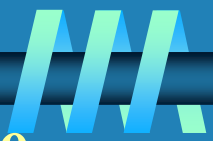
If $X = \{1\}$ and $\overline{X} = \{2,3,4,5,6\}$, $C(X,\overline{X}) = \{(1,2), (1,4)\}$, $c = 5$

If $X = \{1,2,3,4,5\}$ and $\overline{X} = \{6\}$, $C(X,\overline{X}) = \{(3,6), (5,6)\}$, $c = 6$

If $X = \{1,2,4\}$ and $\overline{X} = \{3,5,6\}$, $C(X,\overline{X}) = \{(2,3), (2,5), (4,3)\}$, $c = 9$

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Max-Flow Min-Cut Theorem



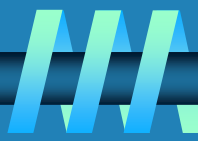
- The value of maximum flow in a network is equal to the capacity of its minimum cut

- The shortest augmenting path algorithm yields both a maximum flow and a minimum cut:
  - maximum flow is the final flow produced by the algorithm
  - minimum cut is formed by all the edges from the labeled vertices to unlabeled vertices on the last iteration of the algorithm
  - all the edges from the labeled to unlabeled vertices are full, i.e., their flow amounts are equal to the edge capacities, while all the edges from the unlabeled to labeled vertices, if any, have zero flow amounts on them



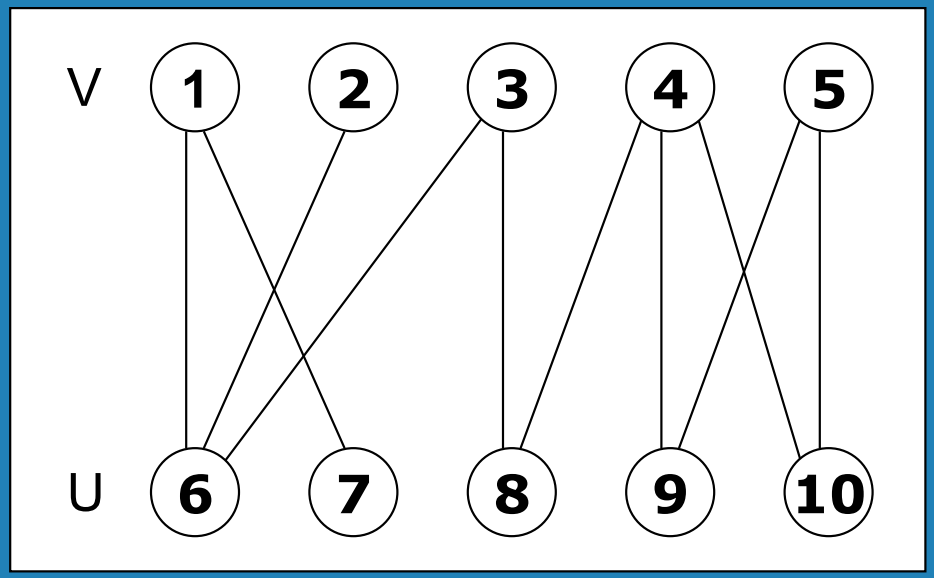
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

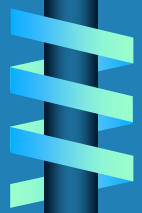
Bipartite Graphs



***Bipartite graph*: a graph whose vertices can be partitioned into two disjoint sets V and U, not necessarily of the same size, so that every edge connects a vertex in V to a vertex in U**

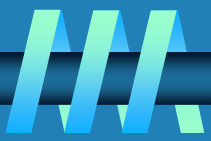
A graph is bipartite if and only if it does not have a cycle of an odd length



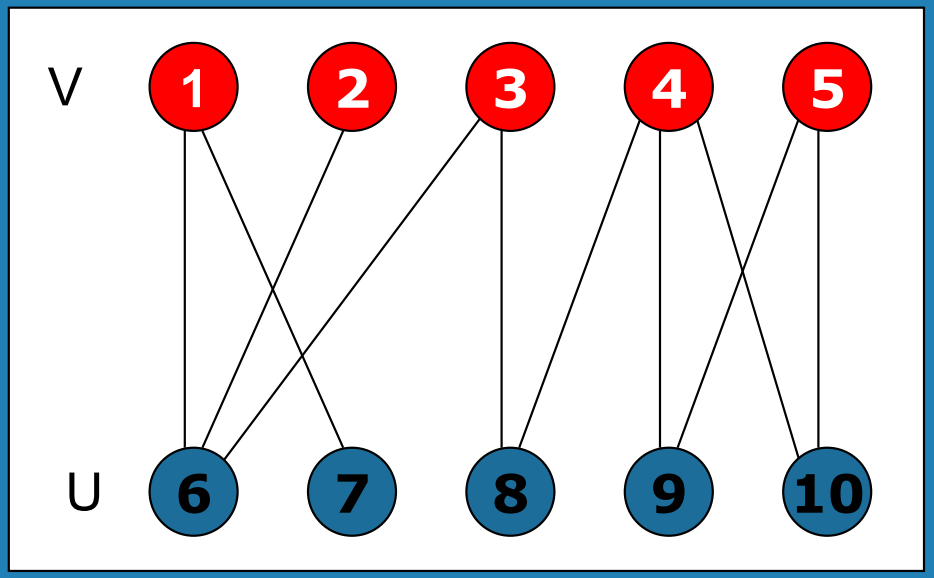


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Bipartite Graphs (cont.)



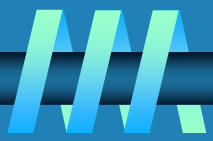
A bipartite graph is *2-colorable*: the vertices can be colored in two colors so that every edge has its vertices colored differently





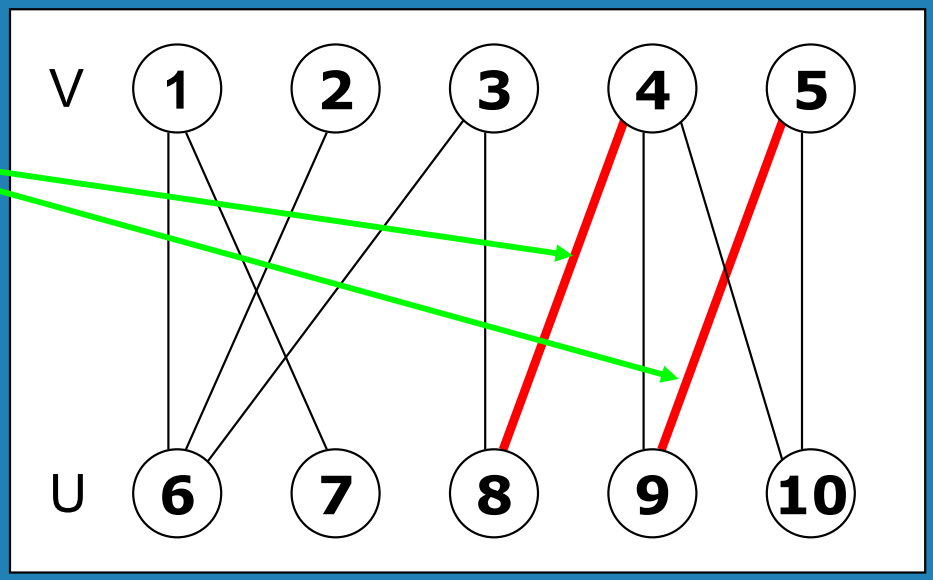
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Matching in a Graph



A *matching* in a graph is a subset of its edges with the property that no two edges share a vertex

a matching
in this graph
M = {(4,8), (5,9)}



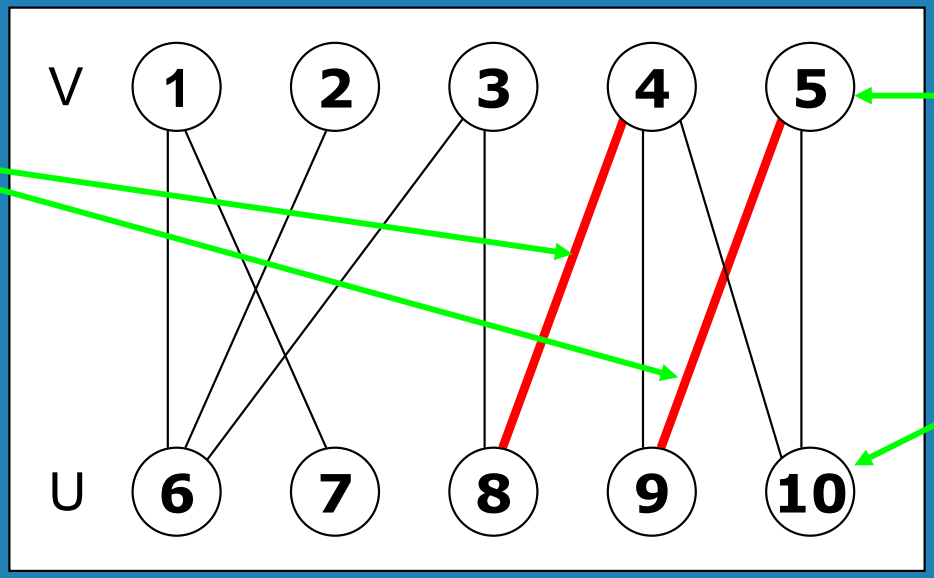
A *maximum* (or *maximum cardinality*) *matching* is a matching with the largest number of edges

- **always exists**
- **not always unique**



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Free Vertices and Maximum Matching



A matching in this graph (M)

A matched vertex

A free vertex

For a given matching M, a vertex is called *free* (or *unmatched*) if it is not an endpoint of any edge in M; otherwise, a vertex is said to be *matched*

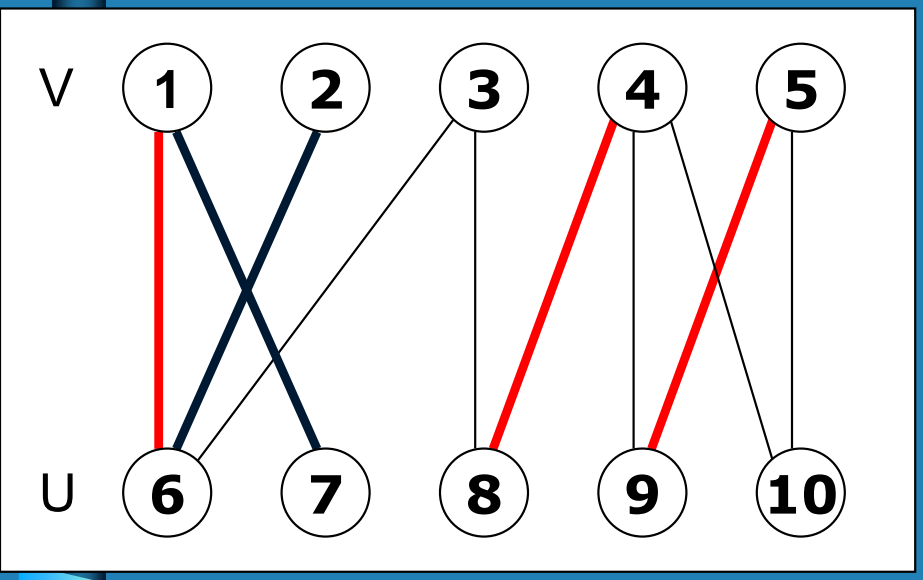
- **If every vertex is matched, then M is a maximum matching**
- **If there are unmatched or free vertices, then M may be able to be improved**
- **We can immediately increase a matching by adding an edge connecting two free vertices (e.g., (1,6) above)**

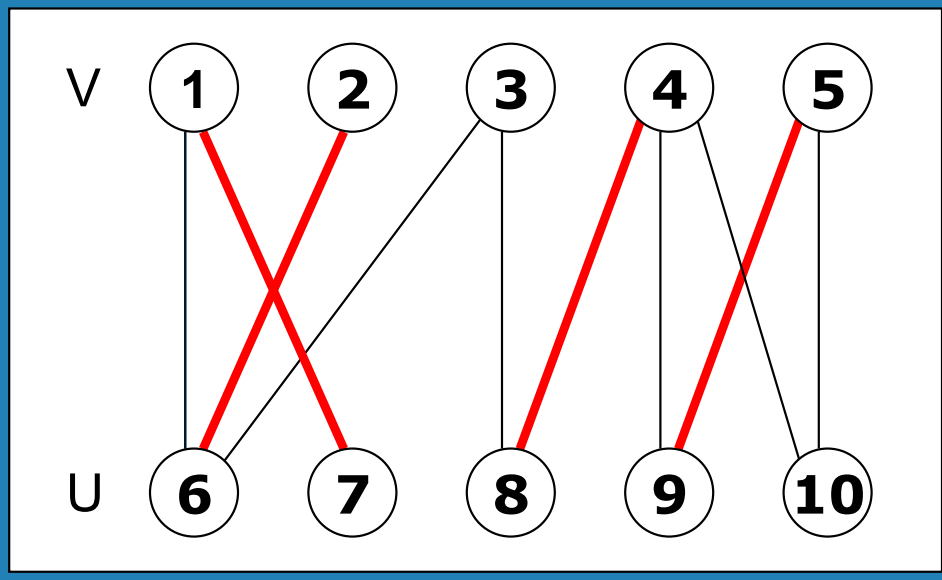
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Augmenting Paths and Augmentation

An *augmenting path* for a matching M is a path from a free vertex in V to a free vertex in U whose edges alternate between edges not in M and edges in M

- The length of an augmenting path is always odd
- Adding to M the odd numbered path edges and deleting from it the even numbered path edges increases the matching size by 1 (*augmentation*)
- One-edge path between two free vertices is special case of augmenting path

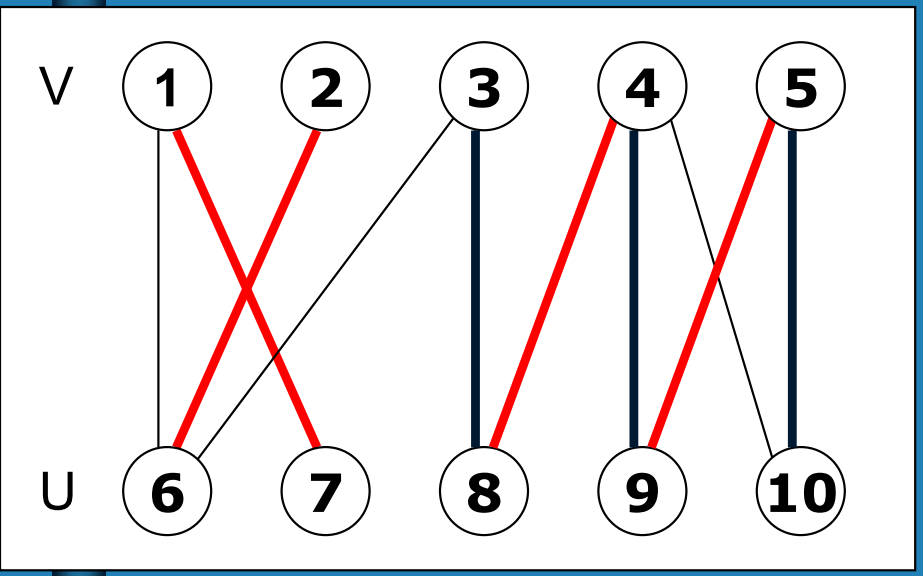


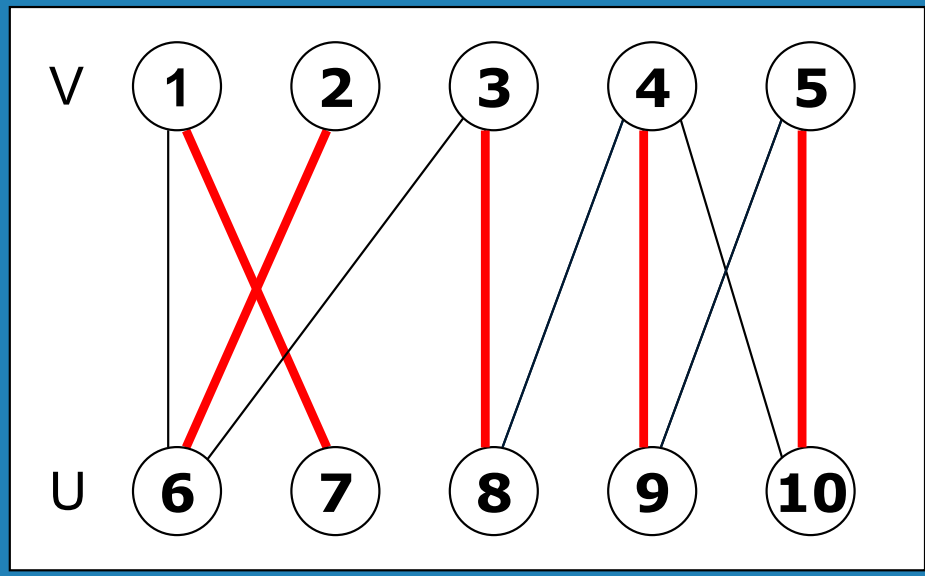


Augmentation along path 2,6,1,7

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Augmenting Paths (another example)



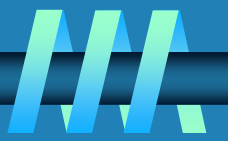


Augmentation along
3, 8, 4, 9, 5, 10

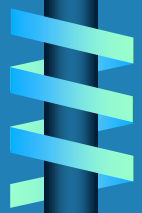
- **Matching on the right is maximum (*perfect* matching)**
- **<u>Theorem</u> A matching M is maximum if and only if there exists no augmenting path with respect to M**

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Augmenting Path Method (template)



- Start with some initial matching
  - e.g., the empty set
- Find an augmenting path and augment the current matching along that path
  - e.g., using breadth-first search like method
- When no augmenting path can be found, terminate and return the last matching, which is maximum



A. Levitin “Introduction to the Design & Analysis of Algorithms,” 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

BFS-based Augmenting Path Algorithm

- Initialize queue Q with all free vertices in one of the sets (say V)

- While Q is not empty, delete front vertex *w* and label every unlabeled vertex *u* adjacent to *w* as follows:

  Case 1 (*w* is in V)

  If *u* is free, augment the matching along the path ending at *u* by moving backwards until a free vertex in V is reached. After that, erase all labels and reinitialize Q with all the vertices in V that are still free

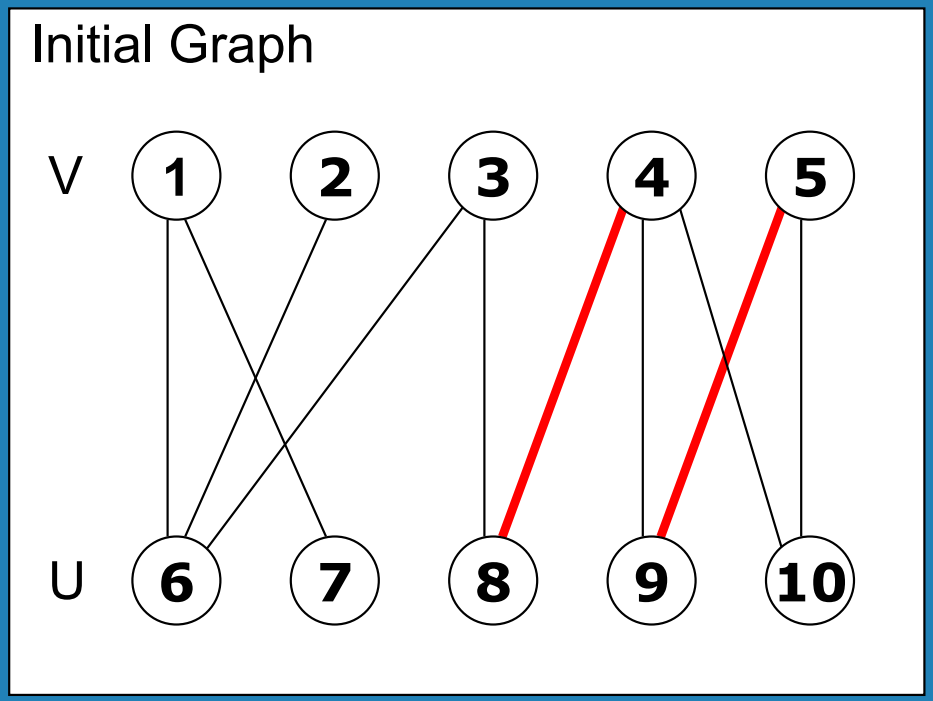
  If *u* is matched (not with *w*), label *u* with *w* and enqueue *u*

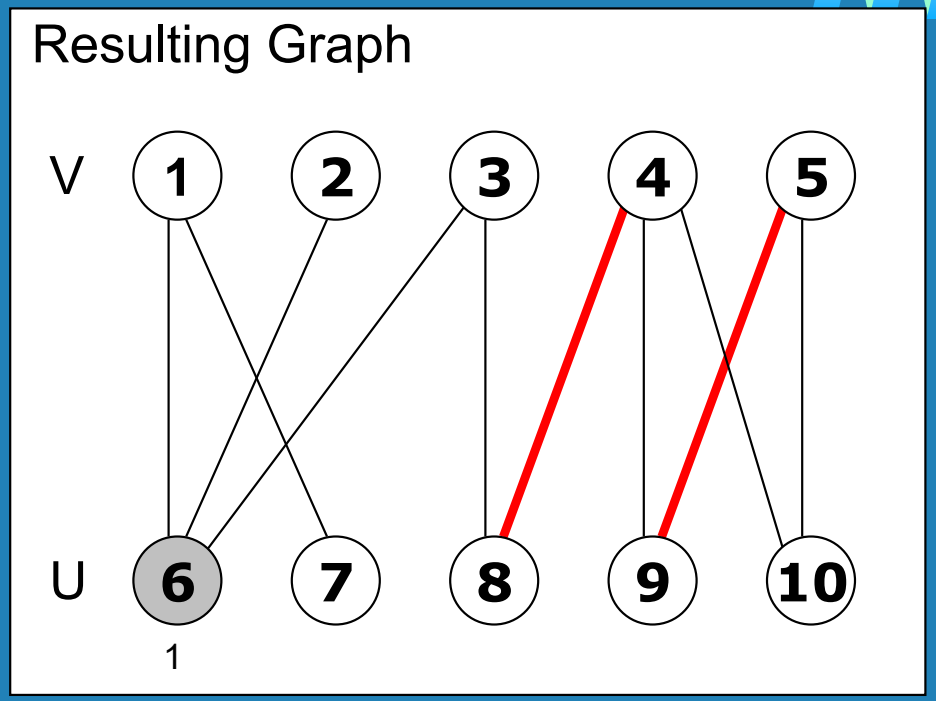
  Case 2 (*w* is in U) Label its matching mate *v* with *w* and enqueue *v*

- After Q becomes empty, return the last matching, which is maximum

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (revisited)





Queue: 1 2 3

Queue: 1 2 3

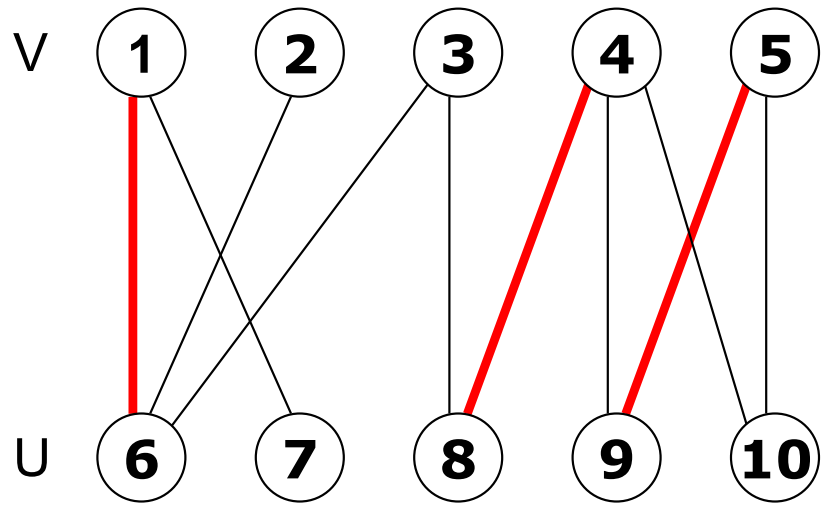
Augment from 6

Each vertex is labeled with the vertex it was reached from. Queue deletions are indicated by arrows. The free vertex found in U is shaded and labeled for clarity; the new matching obtained by the augmentation is shown on the next slide.

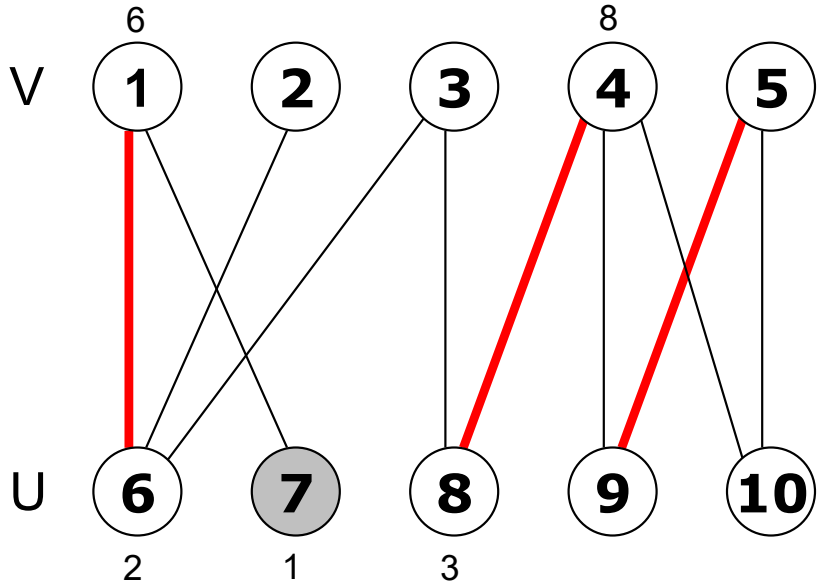
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)

Initial Graph



Resulting Graph



Queue: 2 3

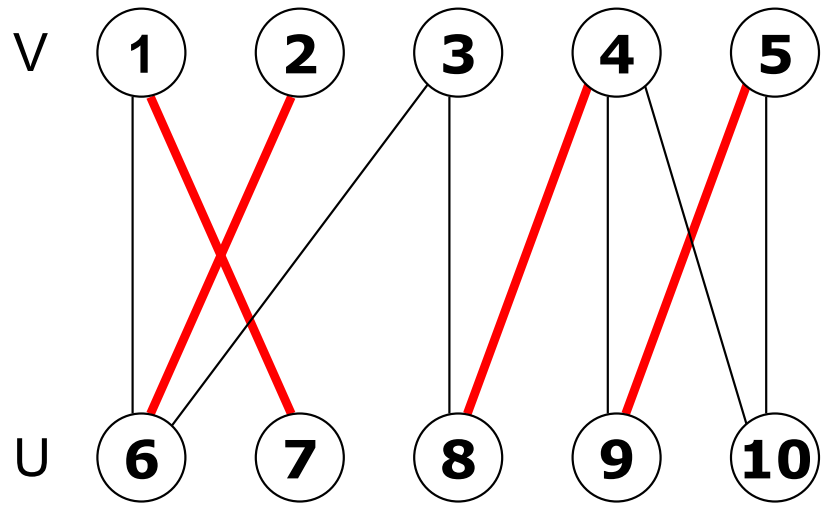
Queue: 2 3 6 8 1 4

Augment from 7

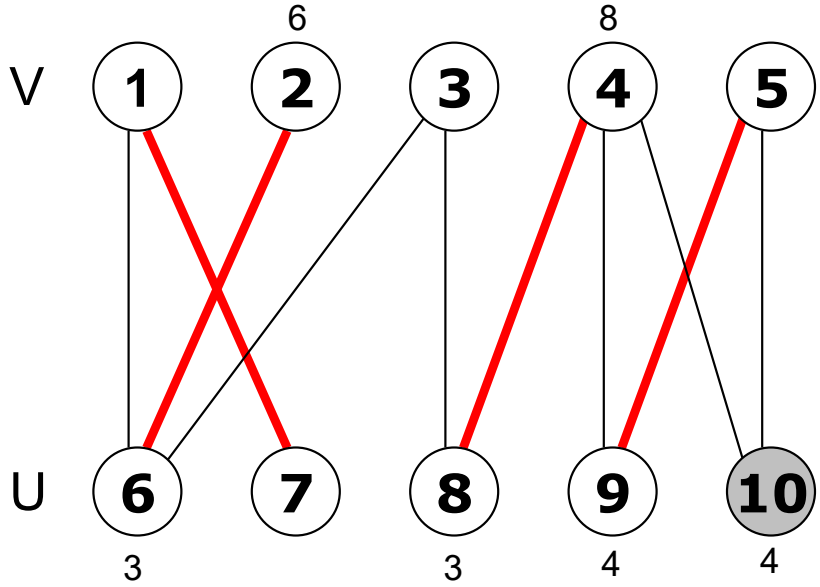
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)

Initial Graph



Resulting Graph



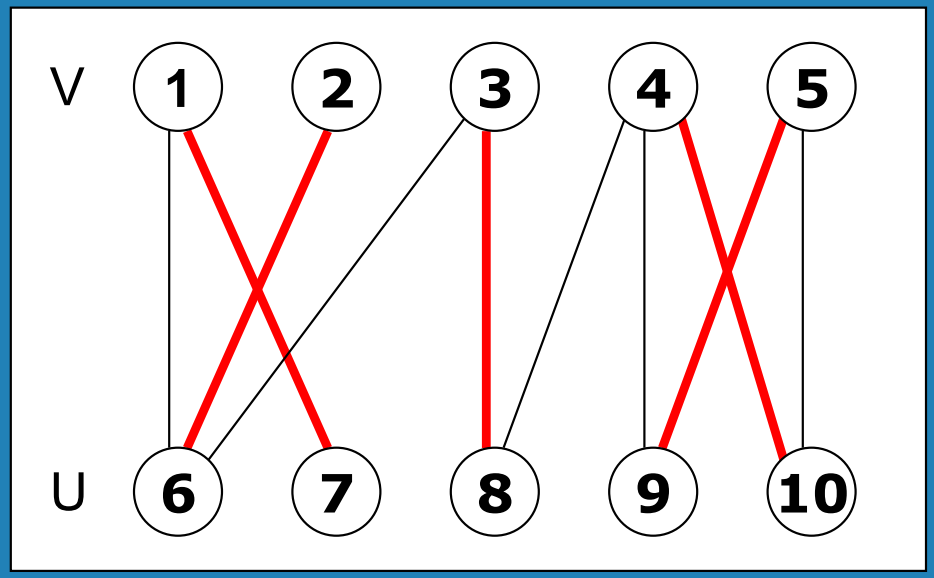
Queue: 3

Queue: 3 6 8 2 4 9

Augment from 10

A. Levitin “Introduction to the Design & Analysis of Algorithms,” 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: maximum matching found



maximum matching

- This matching is maximum since there are no remaining free vertices in V (the queue is empty)
- Note that this matching differs from the maximum matching found earlier

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

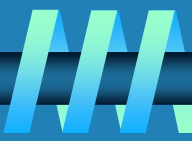
Maximum-matching algorithm for bipartite graphs

Input: A bipartite graph $G = \langle V, U, E \rangle$
Output: A maximum-cardinality matching M in the input graph

```
initialize set M of edges with some valid matching (e.g., the empty set)
initialize queue Q with all the free vertices in V (in any order)
while not Empty(Q) do
    w ← Front(Q); Dequeue(Q)
    if w ∈ V
        for every vertex u adjacent to w do
            if u is free
                //augment
                M ← M ∪ (w, u)
                v ← w
                while v is labeled do
                    u ← vertex indicated by v's label;   M ← M − (v, u)
                    v ← vertex indicated by u's label;   M ← M ∪ (v, u)
                remove all vertex labels
                reinitialize Q with all free vertices in V
                break //exit the for loop
            else //u is matched
                if (w, u) ∉ M and u is unlabeled
                    label u with w
                    Enqueue(Q, u)
    else //w ∈ U (and matched)
        label the mate v of w with "w"
        Enqueue(Q, v)
return M //current matching is maximum
```

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Notes on Maximum Matching Algorithm



- Each iteration (except the last) matches two free vertices (one each from V and U). Therefore, the number of iterations cannot exceed $\lfloor n/2 \rfloor + 1$, where n is the number of vertices in the graph. The time spent on each iteration is in $O(n+m)$, where m is the number of edges in the graph. Hence, the time efficiency is in $O(n(n+m))$

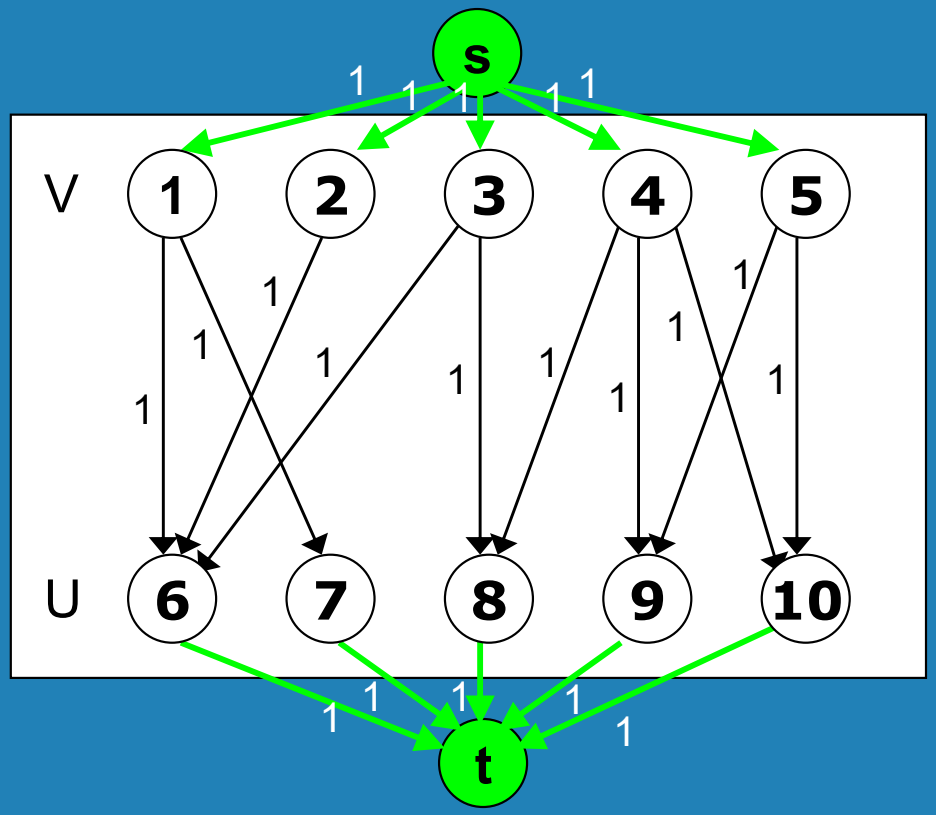
- This can be improved to $O(\sqrt{n}(n+m))$ by combining multiple iterations to maximize the number of edges added to matching M in each search

- Finding a maximum matching in an arbitrary graph is much more difficult, but the problem was solved in 1965 by Jack Edmonds



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

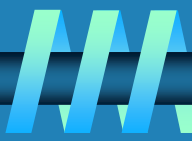
Conversion to Max-Flow Problem



- Add a source and a sink, direct edges (with unit capacity) from the source to the vertices of V and from the vertices of U to the sink
- Direct all edges from V to U with unit capacity

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Stable Marriage Problem



There is a set Y = $\{m_1,\ldots,m_n\}$ of n men and a set X = $\{w_1,\ldots,w_n\}$ of n women. Each man has a ranking list of the women, and each woman has a ranking list of the men (with no ties in these lists).

A *marriage matching* M is a set of n pairs (m_i, w_j).

A pair (m, w) is said to be a *blocking pair* for matching M if man m and woman w are not matched in M but prefer each other to their mates in M.

A marriage matching M is called *stable* if there is no blocking pair for it; otherwise, it's called *unstable*.

The *stable marriage problem* is to find a stable marriage matching for men's and women's given preferences.



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Instance of the Stable Marriage Problem

An instance of the stable marriage problem can be specified either by two sets of preference lists or by a ranking matrix, as in the example below.

men's preferences

	1st	2nd	3rd
Bob:	Lea	Ann	Sue
Jim:	Lea	Sue	Ann
Tom:	Sue	Lea	Ann

women's preferences

	1st	2nd	3rd
Ann:	Jim	Tom	Bob
Lea:	Tom	Bob	Jim
Sue:	Jim	Tom	Bob

ranking matrix

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

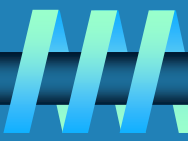
(Bob, Lea) is a blocking pair.

{(Bob, Ann) (Jim, Lea) (Tom, Sue)} is unstable

{(Bob, Ann) (Jim, Sue) (Tom, Lea)} is stable

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Stable Marriage Algorithm (Gale-Shapley)



Step 0 Start with all the men and women being free

Step 1 While there are free men, arbitrarily select one of them and do the following:

Proposal The selected free man *m* proposes to *w*, the next woman on his preference list

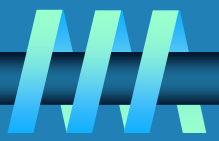
Response If *w* is free, she accepts the proposal to be matched with *m*. If she is not free, she compares *m* with her current mate. If she prefers *m* to him, she accepts *m*'s proposal, making her former mate free; otherwise, she simply rejects *m*'s proposal, leaving *m* free

Step 2 Return the set of *n* matched pairs



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example



Free men:
Bob, Jim, Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Bob proposed to Lea
Lea accepted

Free men:
Jim, Tom

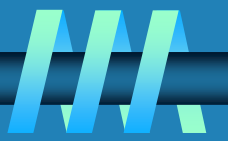
	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Jim proposed to Lea
Lea rejected



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)



Free men:
Jim, Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Jim proposed to Sue
Sue accepted

Free men:
Tom

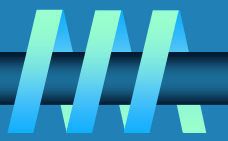
	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Tom proposed to Sue
Sue rejected



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)



Free men:
Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Tom proposed to Lea
Lea replaced Bob with Tom

Free men:
Bob

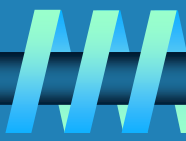
	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Bob proposed to Ann
Ann accepted

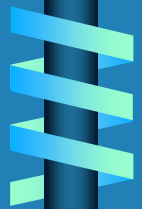


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Analysis of the Gale-Shapley Algorithm



- The algorithm terminates after no more than n^2 iterations with a stable marriage output
- The stable matching produced by the algorithm is always *man-optimal*: each man gets the highest rank woman on his list under any stable marriage. One can obtain the *woman-optimal* matching by making women propose to men
- A man (woman) optimal matching is unique for a given set of participant preferences
- The stable marriage problem has practical applications such as matching medical-school graduates with hospitals for residency training



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 10 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.